Machine Learning Mastery

PJ'S ACADEMY

Complete Course Book

🤖 Machine Learning Mastery (Deep) — PJ's Academy

From "what is a model?" to building, evaluating, and deploying real ML systems — explained so a complete beginner truly understands, not just copies code.

Most ML courses either drown you in math or hand you `model.fit()` with no understanding. This one builds **intuition first**, then math, then code, then a real project — every single topic.

> Prerequisite: basic Python (variables, loops, functions) and basic SQL helps. We teach the ML from zero. Math is explained in plain English before any formula.

🎯 Who This Is For

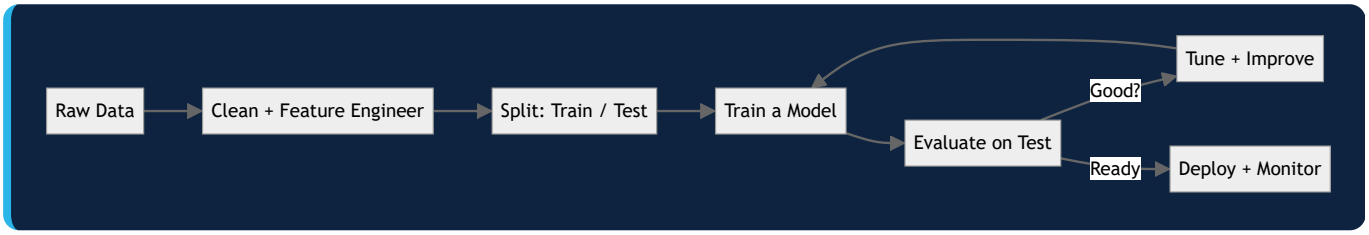
- Beginners who want to *understand* ML, not just run notebooks
- Analysts/developers moving into data science
- Anyone prepping for ML interviews or building AI products

📚 Course Structure — 10 Modules

Module	Topic	You'll Master
01	What ML Really Is	Types, the ML workflow, key mental models
02	Data & Features	Cleaning, encoding, scaling, feature engineering
03	Regression	Linear, polynomial, regularization — predicting numbers
04	Classification	Logistic, kNN, Naive Bayes, decision trees
05	Ensembles	Random Forest, Gradient Boosting, XGBoost
06	Evaluation	Metrics, cross-validation, bias-variance, overfitting
07	Unsupervised	Clustering (K-Means), PCA, dimensionality reduction
08	Tuning & Selection	Feature selection, hyperparameter tuning, pipelines
09	Neural Networks	The intuition + a first network from scratch & Keras
10	ML in Production	Deployment, monitoring, drift, real-world pitfalls

Plus: **15 hands-on projects** and a **practice/quiz bank**.

🧠 The Big Picture (the mental model for the whole course)



Every module fits somewhere in this loop. Keep coming back to this picture.

🧰 Tools You'll Use

`Python` · `NumPy` · `Pandas` · `scikit-learn` · `Matplotlib` · `XGBoost` · `Keras/TensorFlow` (Module 9)

```
pip install numpy pandas scikit-learn matplotlib seaborn xgboost tensorflow
```

💡 How To Learn This

1. Read the **intuition** section — don't skip to code.
2. Run every code example and change the numbers to see what happens.
3. Do the module exercises.
4. Build the module's project before moving on.

🏆 Outcome

You'll be able to frame a problem as ML, prepare data, choose and train the right model, evaluate it honestly, tune it, and deploy it — with a portfolio of 15 projects to prove it.

Course: 🤖 Machine Learning Mastery — PJ's Academy · hello@pjsacademy.com

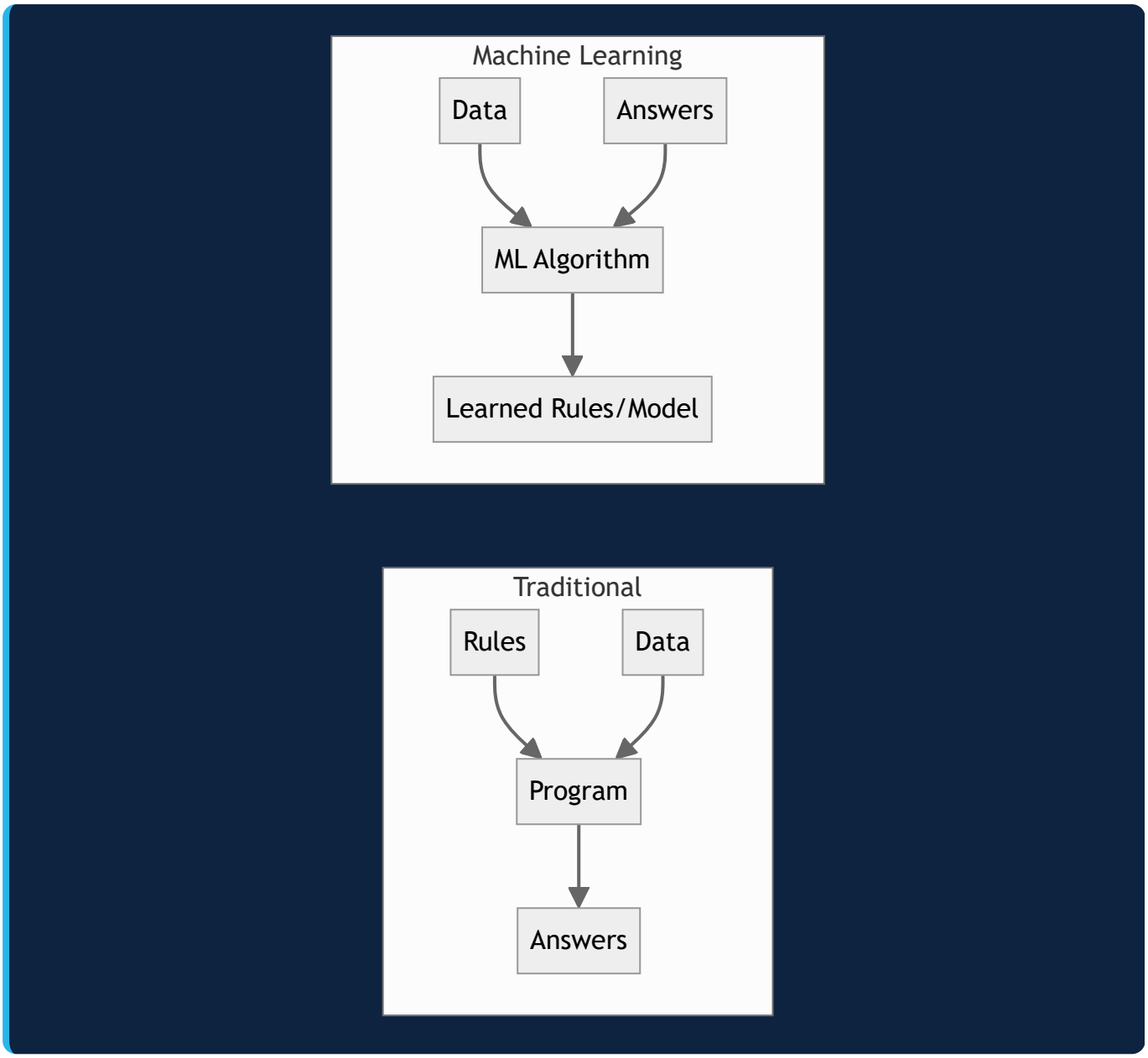
Module 01 — What Machine Learning Really Is

> Before a single line of code: the mental models that make everything else click.

1.1 The One-Sentence Definition

Machine Learning is teaching a computer to find patterns in data so it can make predictions on new data it hasn't seen.

Traditional programming: you write the **rules**, the computer applies them. Machine learning: you give the computer **examples**, and it figures out the rules itself.



Example: To detect spam the old way, you'd write "if email contains 'lottery' → spam." That breaks instantly. ML instead learns from thousands of labelled emails what spam *looks like*, including patterns you'd never think to write.

1.2 The Three Types of ML

Supervised Learning (most of this course)

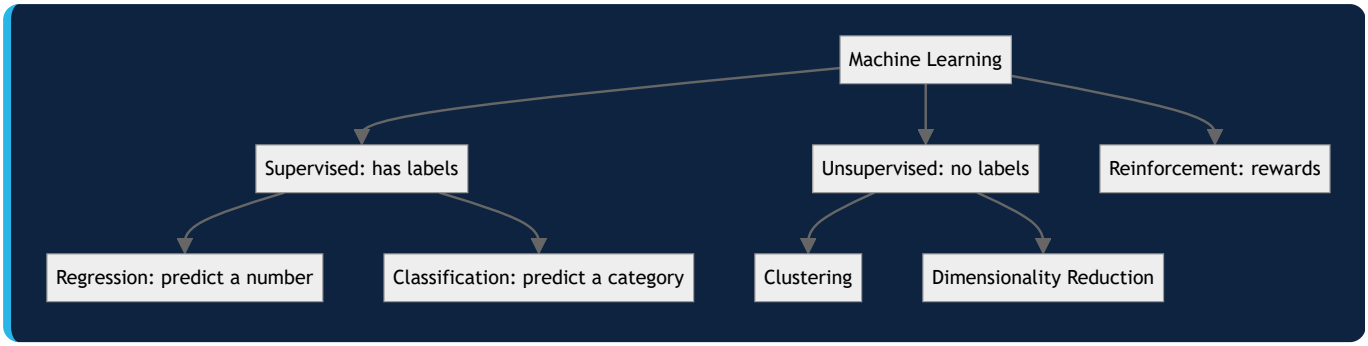
You have **labelled data** — examples with the "right answer." The model learns the mapping from input → output. - **Regression** — predict a **number** (house price, temperature, sales). - **Classification** — predict a **category** (spam/not-spam, disease/healthy, which digit).

Unsupervised Learning

No labels — the model finds **structure** on its own. - **Clustering** — group similar things (customer segments). - **Dimensionality reduction** — compress features (PCA).

Reinforcement Learning

An agent learns by **trial and error** with rewards (game AI, robotics). Not our focus here.



1.3 Core Vocabulary (learn these once, use them forever)

Term	Plain meaning
Feature (X)	An input column — what we know (age, income, size)
Label / Target (y)	The answer we want to predict
Sample / Instance	One row — one example
Model	The learned pattern that maps X → y
Training	The process of learning the pattern from data
Prediction / Inference	Using the trained model on new data
Parameters	Numbers the model learns (e.g., line slope)
Hyperparameters	Settings *you* choose before training (e.g., tree depth)

1.4 The Golden Rule: Train / Test Split

If you test a model on the same data it learned from, of course it does well — it memorised. That tells you nothing about new data.

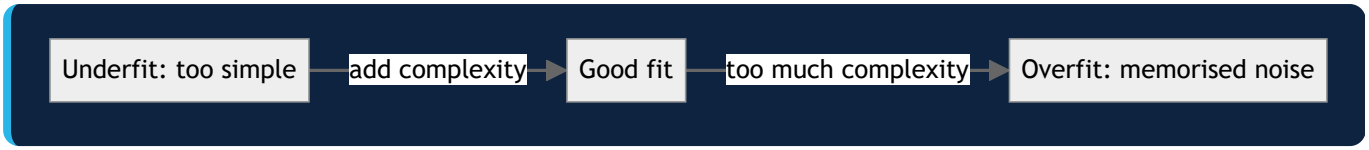
So we **split**: train on ~80%, test on the held-out ~20% it never saw. Test performance = the honest estimate of real-world performance.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# train on X_train/y_train, evaluate on X_test/y_test — NEVER peek at test during training
```

> ⚠️ **Data leakage** — letting any test information sneak into training — is the #1 way beginners get "amazing" scores that collapse in production. Guard the test set like a secret exam.

1.5 Overfitting vs Underfitting (the central tension)

- **Underfitting** — model too simple, misses the pattern. Bad on both train and test.
- **Overfitting** — model too complex, **memorises** training data (including noise). Great on train, bad on test.
- **Just right** — captures the real pattern, generalises to new data.



The entire craft of ML is walking this line — Module 06 gives you the tools.

1.6 The Standard ML Workflow

```
1. Define the problem   → regression or classification? what's the target?
2. Get & explore data   → understand it (EDA)
3. Prepare data         → clean, encode, scale, feature-engineer
4. Split                → train / test
5. Choose & train model → start simple
6. Evaluate             → right metric on the test set
7. Tune & iterate       → improve honestly
8. Deploy & monitor     → real world, watch for drift
```

You'll repeat this loop in every project. Memorise it.

1.7 When NOT to Use ML

ML isn't magic. Skip it when: - A simple rule or formula works (don't ML a tax calculation). - You have very little data (ML needs examples). - You can't tolerate being wrong sometimes (ML is probabilistic). - The cost of errors is life-critical without human oversight.

Good engineers reach for the simplest thing that works.

✅ Key Takeaways

1. ML learns **rules from examples** instead of you writing them.
2. **Supervised** (labelled) = regression (numbers) + classification (categories); **unsupervised** finds structure.
3. Always **train/test split** — test performance is the only honest measure.
4. **Data leakage** silently inflates scores — protect the test set.
5. The game is balancing **underfitting vs overfitting**.
6. Follow the **8-step workflow** every time.

🏋️ Exercises

1. For each, say regression or classification: predicting tomorrow's temperature; whether a transaction is fraud; a house's price; a tumour's type.
2. In your own words, explain overfitting to a 12-year-old using an analogy.
3. Why is it cheating to evaluate on training data? Explain in 2 sentences.

Next: Module 02 — Data & Features →

Module 02 — Data & Feature Engineering

> "Garbage in, garbage out." Models are only as good as the data you feed them. This is where 80% of real ML work happens.

2.1 Why Data Prep Is the Real Job

A fancy algorithm on bad data loses to a simple algorithm on well-prepared data — every time. Real data is messy: missing values, wrong types, different scales, text where you need numbers. Fixing that is the job.

2.2 Exploratory Data Analysis (EDA) First

Before modelling, **look** at your data:

```
import pandas as pd
df = pd.read_csv("data.csv")
df.head()          # first rows — what does it look like?
df.info()          # types + non-null counts
df.describe()      # min/max/mean/std of numeric columns
df.isnull().sum()  # missing values per column
df['target'].value_counts()  # class balance (for classification)
```

Ask: What's the target? Which features look useful? What's missing? Any weird values?

2.3 Handling Missing Values

Options, in rough order of preference:

```
# 1. Drop rows/columns if few and non-critical
df.dropna(subset=['age'])

# 2. Impute numeric with median (robust to outliers)
df['age'] = df['age'].fillna(df['age'].median())

# 3. Impute categorical with the mode (most common)
df['city'] = df['city'].fillna(df['city'].mode()[0])

# 4. Smarter: sklearn imputers (fit on TRAIN only!)
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy='median')
X_train_imp = imp.fit_transform(X_train)
X_test_imp  = imp.transform(X_test)   # transform only — no re-fitting
```

> ⚠️ **Fit imputers/scalers on the training set only, then apply to test.** Fitting on all data leaks test information.

2.4 Encoding Categorical Variables

Models need numbers, not text. Two main tools:

One-Hot Encoding — for categories with no order (city, colour). Each category becomes a 0/1 column.

```
pd.get_dummies(df, columns=['city'])   # city_Mumbai, city_Delhi, ...
```

Ordinal/Label Encoding — for ordered categories (low < medium < high).

```
df['size'] = df['size'].map({'S': 0, 'M': 1, 'L': 2})
```

> Don't label-encode unordered categories with integers — the model wrongly assumes 2 > 1 > 0 has meaning.

2.5 Feature Scaling

Features on wildly different scales (age 0–100 vs income 0–1,000,000) confuse distance- and gradient-based models (kNN, SVM, neural nets, linear models). Put them on a comparable scale.

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
# Standardization: mean 0, std 1 (most common)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)   # fit on train
X_test_s  = scaler.transform(X_test)        # apply to test
```

- **StandardScaler** (z-score): centres to mean 0, std 1. Default choice.
- **MinMaxScaler**: squashes to [0, 1]. Good for bounded/image data.
- **Tree-based models** (Random Forest, XGBoost) **don't need scaling** — they split on thresholds, scale-invariant.

2.6 Feature Engineering — the Highest-Leverage Skill

Creating better features from existing ones often beats any model upgrade.

```
# Ratios and interactions
df['rooms_per_person'] = df['rooms'] / df['occupants']
df['price_per_sqft']   = df['price'] / df['area']

# Date features
df['order_date'] = pd.to_datetime(df['order_date'])
df['dayofweek']  = df['order_date'].dt.dayofweek
df['is_weekend'] = (df['dayofweek'] >= 5).astype(int)
df['month']      = df['order_date'].dt.month

# Binning a continuous variable
df['age_group'] = pd.cut(df['age'], bins=[0,18,35,60,100],
                         labels=['teen','young','adult','senior'])

# Text length, counts, flags
df['name_length'] = df['name'].str.len()
```

Domain knowledge shines here: *what would a human expert look at?* Encode that.

2.7 Outliers

Extreme values can distort models (especially linear ones).

```
# Detect with the IQR rule
Q1, Q3 = df['income'].quantile([0.25, 0.75])
IQR = Q3 - Q1
low, high = Q1 - 1.5*IQR, Q3 + 1.5*IQR
outliers = df[(df['income'] < low) | (df['income'] > high)]
```

Options: remove (if errors), cap/winsorize (clip to a limit), or keep (if genuine — e.g. fraud *is* the outlier). Decide with domain sense, not reflex.

2.8 Putting It Together with a Pipeline

Pipelines bundle preprocessing + model so the same steps apply consistently and leak-free:

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

numeric = ['age', 'income']
categorical = ['city']

pre = ColumnTransformer([
    ('num', Pipeline([('imp', SimpleImputer(strategy='median')),
                      ('sc', StandardScaler())]), numeric),
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical),
])
# pre.fit_transform(X_train) then pre.transform(X_test) — or drop into a full model Pipeline
```

> Pipelines are the professional standard — they make preprocessing reproducible and prevent leakage automatically.

✅ Key Takeaways

1. **EDA first** — understand data before modelling.
2. **Impute** missing values (median/mode); fit on train only.
3. **One-hot** unordered categories, **ordinal-encode** ordered ones.
4. **Scale** features for distance/gradient models; trees don't need it.
5. **Feature engineering** (ratios, dates, bins) is the highest-leverage move.
6. Use **pipelines** to apply steps consistently and avoid leakage.

🏋️ Exercises

1. Take any CSV, run the 5 EDA commands, and write 3 things you learned.
2. One-hot encode a categorical column and ordinal-encode an ordered one. Explain why each choice.
3. Engineer 3 new features from a dataset and argue why each might help.
4. Build a `ColumnTransformer` that imputes + scales numerics and one-hots categoricals.

Next: Module 03 — Regression →

Module 03 — Regression: Predicting Numbers

> Your first real models. Regression predicts a continuous number — price, temperature, demand. We build intuition, then math, then code.

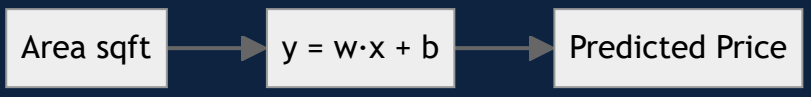
3.1 The Idea: Fit a Line Through Data

Simple linear regression finds the **straight line** that best fits your data:

```
y = w·x + b
```

- `y` = prediction (e.g., house price)
- `x` = feature (e.g., area)
- `w` = weight/slope (how much price changes per unit area)
- `b` = bias/intercept (baseline price at x=0)

"Learning" = finding the `w` and `b` that make the line fit best.



3.2 What "Best Fit" Means — the Cost Function

"Best" = smallest total error. We measure error as **Mean Squared Error (MSE)**: average of (actual − predicted)^2, squared so positives and negatives don't cancel and big errors are punished more.

$$MSE = (1/n) \cdot \Sigma (y_i - \hat{y}_i)^2$$

Training = adjusting `w` and `b` to minimize MSE. The algorithm (gradient descent) nudges them downhill until the error stops shrinking.

3.3 Your First Model in Code

```python
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)          # learn w and b
pred = model.predict(X_test)         # predict on unseen data

print("Coefficients (w):", model.coef_)
print("Intercept (b):", model.intercept_)
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred)))
print("R²:", r2_score(y_test, pred))
```

3.4 Reading the Metrics

- **RMSE** (Root Mean Squared Error) — average error in the target's units. "On average we're off by ₹X." Lower is better.
- **MAE** (Mean Absolute Error) — like RMSE but less sensitive to big outliers.
- **R^2** (0–1) — fraction of variance explained. 0.85 = "the model explains 85% of the variation." Higher is better; can go negative if the model is worse than guessing the mean.

3.5 Multiple Linear Regression

More features, same idea — now a plane/hyperplane instead of a line:

```
y = w₁x₁ + w₂x₂ + ... + wₙxₙ + b
```

```
# X has multiple columns; scikit-learn handles it identically
model = LinearRegression().fit(X_train, y_train)  # one weight per feature
```

Each weight tells you that feature's effect, holding others constant — useful for **interpretation**, not just prediction.

3.6 Polynomial Regression — Curves, Not Just Lines

When the relationship bends, add polynomial features (x^2, x^3) so a linear model can fit curves:

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

model = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())
model.fit(X_train, y_train)
```

> ⚠️ High degrees overfit fast — a degree-10 polynomial will wiggle through every training point and fail on new data. Start low.

3.7 Regularization — the Cure for Overfitting

Regularization penalizes large weights, keeping the model simpler and more general.

- **Ridge (L2)** — shrinks weights toward zero (rarely exactly zero). Good default.
- **Lasso (L1)** — can push weights to *exactly* zero → automatic **feature selection**.
- **ElasticNet** — a blend of both.

```
from sklearn.linear_model import Ridge, Lasso
ridge = Ridge(alpha=1.0).fit(X_train, y_train)   # alpha = strength of the penalty
lasso = Lasso(alpha=0.1).fit(X_train, y_train)    # bigger alpha = simpler model
```

`alpha` is a **hyperparameter**: too small = overfits, too large = underfits. Tune it (Module 08).

> **Scale your features before Ridge/Lasso** — the penalty is unfair otherwise (big-scale features get penalized more).

3.8 Assumptions & Diagnostics (why a model might mislead)

Linear regression assumes a roughly linear relationship, independent errors, and constant error spread. Check with a **residual plot** (errors vs predictions):

```
import matplotlib.pyplot as plt
residuals = y_test - pred
plt.scatter(pred, residuals); plt.axhline(0, color='red')
plt.xlabel('Predicted'); plt.ylabel('Residual'); plt.show()
```

Residuals should look like random scatter around 0. Patterns (a curve, a funnel) mean the model is missing something — engineer features or use a non-linear model.

3.9 When to Use What

- **Linear/Ridge:** interpretable baseline; relationships roughly linear.
- **Lasso:** many features, want automatic selection.
- **Polynomial:** clear curvature, few features.
- **Tree-based regressors (next modules):** complex non-linear relationships, less prep.

Always start with a **simple linear baseline** — you need something to beat.

✅ Key Takeaways

1. Regression fits `y = w·x + b`; training minimizes **MSE**.
2. Evaluate with **RMSE/MAE** (error size) and **R^2** (variance explained).
3. **Polynomial features** let linear models fit curves — but overfit if too high.
4. **Regularization** (Ridge/Lasso) fights overfitting; Lasso also selects features.
5. **Scale features** before regularized models.
6. **Residual plots** reveal what the model is missing. Always start with a simple baseline.

🏋️ Exercises

1. Train `LinearRegression` on any numeric dataset; report RMSE and R^2.
2. Add `PolynomialFeatures(degree=2)` — does test R^2 improve or overfit?
3. Compare Ridge with `alpha=0.1, 1, 10`. Plot how coefficients shrink.
4. Make a residual plot and interpret it — is the linear assumption OK?

🛠️ Mini-Project

Predict house prices (California Housing, built into sklearn). Baseline linear → add engineered features → Ridge with tuned alpha. Report RMSE improvement at each step.

Next: Module 04 — Classification →

Module 04 — Classification: Predicting Categories

> Predicting *which class* something belongs to — spam/not-spam, disease/healthy, which digit. The most common ML task in industry.

4.1 Regression vs Classification

- Regression predicts a **number** (₹45,000).
- Classification predicts a **category** (spam / not spam), usually via a **probability** (0.92 → spam).

The output is a class label, and often a probability behind it that you threshold (e.g., ≥ 0.5 → positive).

4.2 Logistic Regression (don't let the name fool you — it's classification)

It predicts a **probability** by squashing a linear score through the **sigmoid** function (an S-curve that maps any number to 0–1):

```
score = w·x + b   →   probability = sigmoid(score) = 1 / (1 + e^-score)
```

If probability ≥ 0.5 → class 1, else class 0.

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
pred   = model.predict(X_test)            # class labels
proba  = model.predict_proba(X_test)[:,1] # probability of class 1
```

> Great interpretable baseline for any classification problem. Scale features first.

4.3 k-Nearest Neighbours (kNN) — "you are who your neighbours are"

To classify a new point, look at the **k closest** training points and take the majority vote.

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
```

- **Must scale features** (it's distance-based).
- `k` too small → noisy/overfit; too large → oversmoothed. Tune it.
- Simple and intuitive, but slow on big data (compares to everything).

4.4 Naive Bayes — fast, great for text

Uses probability (Bayes' theorem) and "naively" assumes features are independent. Surprisingly strong for **text/spam classification**.

```
from sklearn.naive_bayes import MultinomialNB
nb = MultinomialNB().fit(X_train, y_train)  # X often word counts / TF-IDF
```

Blazing fast, works well with many features (words). A classic spam-filter engine.

4.5 Decision Trees — human-readable rules

A tree of yes/no questions that split the data toward pure groups:

```
Is income > 50k?
├─ Yes → Is age > 30? → ...
└─ No  → Class: Reject
```

```
from sklearn.tree import DecisionTreeClassifier
tree = DecisionTreeClassifier(max_depth=4, random_state=42)
tree.fit(X_train, y_train)
```

- **No scaling needed**; handles non-linear patterns; easy to explain.
- **Overfits badly if unpruned** — always limit `max_depth` / `min_samples_leaf`.
- Single trees are unstable → we fix this with **ensembles** (Module 05).

```
# See the rules
from sklearn.tree import export_text
print(export_text(tree, feature_names=list(X.columns)))
```

4.6 Evaluating Classification (accuracy is a trap)

Accuracy = % correct. But on imbalanced data it lies: if 99% of transactions are legit, a model that says "always legit" is 99% accurate and 100% useless.

Use the **confusion matrix** and its metrics:

```
              Predicted +   Predicted −
Actual +          TP            FN
Actual −          FP            TN
```

- **Precision** = TP / (TP + FP) — "of those I flagged positive, how many really are?" (cost of false alarms)
- **Recall** = TP / (TP + FN) — "of all real positives, how many did I catch?" (cost of misses)
- **F1** = harmonic mean of precision & recall — one balanced number.
- **ROC-AUC** — ranks how well the model separates classes across all thresholds (0.5 = random, 1.0 = perfect).

```
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))
print("ROC-AUC:", roc_auc_score(y_test, proba))
```

4.7 Precision vs Recall — a business decision

You usually trade one for the other by moving the probability **threshold**: - **Cancer screening** → maximise **recall** (never miss a case; false alarms are acceptable). - **Spam filter** → maximise **precision** (don't send real mail to spam; a little spam getting through is OK).

```
# Custom threshold instead of the default 0.5
pred_custom = (proba >= 0.3).astype(int)   # lower threshold → higher recall
```

4.8 Handling Class Imbalance

When one class is rare (fraud, disease): - `class_weight='balanced'` in the model (penalize mistakes on the rare class more). - **Resampling** — oversample the minority (SMOTE) or undersample the majority. - Evaluate with **PR-AUC / F1 / recall**, never plain accuracy.

```
LogisticRegression(class_weight='balanced', max_iter=1000)
```

4.9 Choosing a Classifier (starting map)

- **Interpretable baseline** → Logistic Regression.
- **Text** → Naive Bayes.
- **Small, need explanation** → Decision Tree.
- **Best accuracy on tabular data** → ensembles (Module 05) — usually the winner.

Start simple, measure, then upgrade.

✅ Key Takeaways

1. Classification predicts a **category**, usually via a **probability** you threshold.
2. **Logistic Regression** = sigmoid over a linear score; strong interpretable baseline.
3. **kNN** (scale it!), **Naive Bayes** (text), **Decision Trees** (readable, limit depth).
4. **Accuracy lies on imbalanced data** — use the **confusion matrix**, precision, recall, F1, ROC-AUC.
5. Trade **precision vs recall** by the threshold, based on business cost.
6. Handle **imbalance** with class weights / resampling and the right metrics.

🏋️ Exercises

1. Train Logistic Regression + a Decision Tree on the same data; compare F1 and ROC-AUC.
2. Print a confusion matrix and explain each cell (TP/FP/FN/TN) in words.
3. Lower the threshold to 0.3 and describe what happens to precision vs recall.
4. On an imbalanced dataset, compare accuracy vs F1 — show why accuracy misleads.

🛠️ Mini-Project

Build a churn classifier (Telco dataset). Logistic baseline → tune threshold for the business goal → handle imbalance with `class_weight='balanced'`. Report precision, recall, F1, and which errors cost more.

Next: Module 05 — Ensembles →

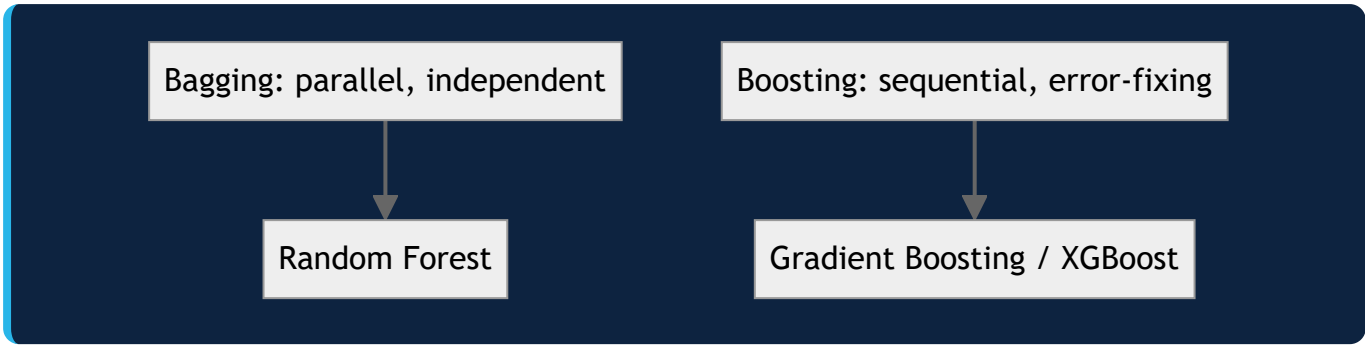
Module 05 — Ensembles: The Models That Win

> One tree is weak. A thousand trees voting together is state-of-the-art on tabular data. This is what wins Kaggle and ships in industry.

5.1 The Core Idea: Wisdom of Crowds

An **ensemble** combines many models so their errors cancel out. A crowd of diverse, decent models beats one "genius" model. Two main flavours:

- **Bagging** (parallel) — train many models on random subsets, average them. Reduces **variance** (overfitting). → **Random Forest**
- **Boosting** (sequential) — each model fixes the previous one's mistakes. Reduces **bias**. → **Gradient Boosting / XGBoost**



5.2 Random Forest — many trees, one vote

Trains many decision trees, each on a random sample of rows **and** a random subset of features, then averages (regression) or votes (classification). The randomness makes trees diverse so their mistakes cancel.

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=300, max_depth=None,
                            max_features='sqrt', random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
```

- **Robust, little tuning, no scaling needed** — a fantastic default.
- Handles non-linearity and interactions automatically.
- Gives **feature importance** for free.

```
import pandas as pd
pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False).head(10)
```

5.3 Gradient Boosting — learn from mistakes

Builds trees **sequentially**; each new tree predicts the **residual errors** of the combined previous trees. Slowly, greedily, it drives error down. Usually the **most accurate** on tabular data.

```
from sklearn.ensemble import GradientBoostingClassifier
gb = GradientBoostingClassifier(n_estimators=300, learning_rate=0.05,
                                max_depth=3, random_state=42)
gb.fit(X_train, y_train)
```

5.4 XGBoost — the industry favourite

A fast, regularized, highly-optimized gradient boosting library. The go-to for tabular competitions and production.

```
import xgboost as xgb
model = xgb.XGBClassifier(
    n_estimators=400, learning_rate=0.05, max_depth=6,
    subsample=0.8, colsample_bytree=0.8,   # row/feature sampling = regularization
    reg_lambda=1.0, random_state=42, eval_metric='logloss')
model.fit(X_train, y_train)
```

Cousins: LightGBM (faster on big data), **CatBoost** (handles categoricals natively). Same idea, different engines.

5.5 The Key Hyperparameters (boosting)

Param	Effect	Rule of thumb
`n_estimators`	number of trees	more + low learning_rate = better but slower
`learning_rate`	how much each tree contributes	lower = more accurate, needs more trees (0.01–0.1)
`max_depth`	tree complexity	3–8; deeper overfits
`subsample` / `colsample_bytree`	randomness	0.7–0.9 to reduce overfitting
`reg_lambda` / `reg_alpha`	penalties	raise to fight overfitting

> The classic combo: **low** `learning_rate` **+ high** `n_estimators` + early stopping.

5.6 Early Stopping — don't waste trees

Stop adding trees once validation performance stops improving:

```
model.fit(X_train, y_train,
          eval_set=[(X_val, y_val)],
          verbose=False)   # newer XGBoost: set early_stopping_rounds in the constructor
```

Prevents overfitting and saves time.

5.7 Bagging vs Boosting — when to use which

- **Random Forest:** safe, fast to get working, hard to overfit, minimal tuning. Great first strong model.
- **XGBoost/LightGBM:** squeeze out the best accuracy, willing to tune. Usually wins.
- Try **both**, compare on the test set with proper metrics.

5.8 Stacking (advanced) — models of models

Train several different models, then a "meta-model" learns to combine their predictions:

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
stack = StackingClassifier(
    estimators=[('rf', rf), ('xgb', model)],
    final_estimator=LogisticRegression())
stack.fit(X_train, y_train)
```

A small extra boost when every fraction of a percent matters.

✅ Key Takeaways

1. Ensembles combine many models so errors cancel — the best tabular approach.
2. **Bagging (Random Forest)** cuts variance; **Boosting (XGBoost)** cuts bias.
3. **Random Forest** = robust default, feature importance, no scaling.
4. **XGBoost/LightGBM** = usually the most accurate; tune `learning_rate` + `n_estimators` + `max_depth`.
5. Use **early stopping** and row/feature **subsampling** to control overfitting.
6. **Stacking** squeezes out the last bit of performance.

🏋️ Exercises

1. Train Random Forest and XGBoost on the same data; compare ROC-AUC.
2. Plot the top-10 feature importances from a Random Forest.
3. Sweep XGBoost `learning_rate` (0.3, 0.1, 0.05) with matching `n_estimators`; observe accuracy vs time.

🛠️ Mini-Project

Take your Module 04 churn model and beat it with XGBoost. Tune the top 3 hyperparameters, use early stopping, and report the F1/ROC-AUC gain over the logistic baseline.

Next: Module 06 — Evaluation →

Module 06 — Evaluation: Trusting Your Model

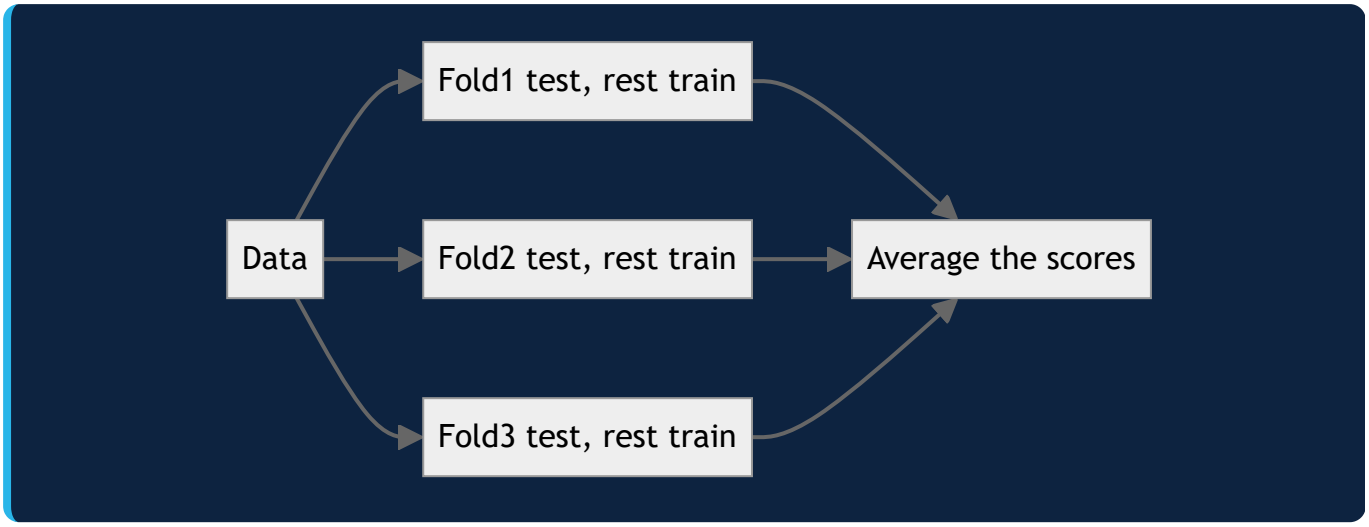
> A model you can't evaluate honestly is a liability. This module is what separates people who *think* their model works from people who *know*.

6.1 The One Sin: Evaluating on Training Data

Training accuracy tells you how well the model memorised — not how it'll do on new data. **Always judge on data the model never saw.** Everything here builds on that.

6.2 Cross-Validation — a more reliable estimate

A single train/test split is one lucky/unlucky draw. **k-fold cross-validation** splits data into k parts, trains on k−1 and tests on the held-out fold, rotating k times, then averages. You get a stable estimate + a sense of variance.




```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='f1')
print(scores.mean(), "±", scores.std())
```

- Use **StratifiedKFold** for classification (keeps class balance in each fold).
- Use **TimeSeriesSplit** for time data (never train on the future!).

6.3 The Bias–Variance Tradeoff (the deep why)

Every model's error = **bias** + **variance** + irreducible noise. - **High bias** (underfit) — too simple, wrong assumptions. Fix: more complex model, more features. - **High variance** (overfit) — too sensitive to training data. Fix: more data, regularization, simpler model, ensembling (bagging).

```
Underfit ◄──────────── sweet spot ────────────► Overfit
(high bias)                                    (high variance)
```

Diagnose with a **learning curve**: plot train vs validation score as you add data. - Both low & close → underfitting (bias). - Big gap (train high, val low) → overfitting (variance).

```
from sklearn.model_selection import learning_curve
# plot train_scores vs val_scores over increasing training size
```

6.4 Regression Metrics (recap + when)

- **RMSE** — punishes big errors; same units as target. Default.
- **MAE** — robust to outliers; "typical" error.
- **R^2** — variance explained (0–1). Good for communicating.
- **MAPE** — error as a %; good for business, but breaks near zero.

6.5 Classification Metrics (recap + the right choice)

Situation	Use
Balanced classes, equal error cost	Accuracy is fine
Imbalanced classes	**F1, PR-AUC, recall** — never plain accuracy
Ranking / threshold-agnostic	**ROC-AUC**
False alarms costly	Precision
Misses costly	Recall

6.6 The Validation Set (three-way split)

When you tune hyperparameters, you peek at the test set repeatedly → you start overfitting to it. Fix: **three splits**.

```
Train (fit params) → Validation (tune hyperparams) → Test (final, touched ONCE)
```

Or use cross-validation for tuning and keep a final held-out test set for the last, honest number.

6.7 Don't Leak — subtle ways test info sneaks in

- Scaling/imputing on the full dataset before splitting. → Fit on train only (use pipelines).
- Feature engineering using target statistics computed over all data.
- Time series: shuffling so future rows train the model.
- Duplicate rows across train and test.

> Leakage produces "amazing" offline scores that **collapse in production.** If a result seems too good, suspect leakage first.

6.8 Beyond a Single Number

- **Confusion matrix** — see *where* it's wrong (which classes get confused).
- **Residual plots** (regression) — see systematic errors.
- **Calibration** — do predicted probabilities match reality? (0.8 should be right ~80% of the time.)
- **Error analysis** — read the actual misclassified examples. This finds more improvements than any tuning.
- **Slice metrics** — check performance per segment (does it fail for one group?). Critical for fairness.

6.9 A Trustworthy Evaluation Checklist

- [ ] Held-out test set, touched once.
- [ ] Right metric for the problem (not just accuracy).

- [ ] Cross-validation for a stable estimate.
- [ ] No leakage (fit preprocessing on train only).
- [ ] Compared against a simple baseline.
- [ ] Looked at *where* it fails, not just the score.

✅ Key Takeaways

1. Judge only on **unseen data**; use **cross-validation** for stable estimates.
2. Error = **bias + variance**; diagnose with learning curves and fix accordingly.
3. Pick the **metric that matches the cost** — accuracy lies on imbalanced data.
4. Use a **train/validation/test** split (or CV + final test) so tuning doesn't corrupt your estimate.
5. **Leakage** is the silent killer — fit preprocessing on train only.
6. Look **beyond one number**: confusion matrix, calibration, error analysis, per-slice metrics.

🏋️ Exercises

1. Run 5-fold cross-validation and report mean ± std. Compare to a single split.
2. Plot a learning curve and diagnose bias vs variance.
3. Deliberately leak (scale before splitting), then do it correctly — compare the scores.
4. For 3 scenarios (fraud, spam, tumour screening), pick the metric to optimize and justify it.

Next: Module 07 — Unsupervised Learning →

Module 07 — Unsupervised Learning: Finding Hidden Structure

> No labels, no "right answer" — just data. The model discovers patterns on its own: groups, structure, compression.

7.1 What "Unsupervised" Means

You have features (X) but **no target (y)**. The goal isn't prediction — it's **understanding**: What natural groups exist? What are the main dimensions of variation? Which points are anomalies?

Two workhorses: **clustering** (grouping) and **dimensionality reduction** (compressing).

7.2 K-Means Clustering

Groups data into **k** clusters by repeatedly: assign each point to the nearest cluster centre, then move each centre to the average of its points — until stable.

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

X_scaled = StandardScaler().fit_transform(X)   # scaling is essential (distance-based)
km = KMeans(n_clusters=4, n_init=10, random_state=42)
labels = km.fit_predict(X_scaled)              # which cluster each point belongs to
```

Choosing k — the Elbow Method: plot within-cluster error (inertia) vs k; the "elbow" where it stops dropping sharply is a good k.

```
inertias = [KMeans(k, n_init=10, random_state=42).fit(X_scaled).inertia_ for k in range(1,10)]
# plot inertias vs k, look for the elbow
```

Also check the **silhouette score** (−1 to 1; higher = better-separated clusters).

> **Always scale before K-Means** — otherwise the largest-scale feature dominates the distances.

7.3 Real Use: Customer Segmentation

The classic business application — group customers by behaviour, then treat each segment differently:

```
# Features: recency, frequency, monetary (RFM)
km = KMeans(n_clusters=4, n_init=10, random_state=42)
df['segment'] = km.fit_predict(StandardScaler().fit_transform(df[['recency','frequency','monetary']]))
# Now: "high-value loyal", "at-risk", "new", "bargain" → tailored marketing
```

7.4 Other Clustering Methods

- **Hierarchical clustering** — builds a tree of nested clusters (dendrogram); no need to pick k upfront.
- **DBSCAN** — density-based; finds arbitrary shapes and labels outliers as noise. Great when clusters aren't round and you don't know k.

```
from sklearn.cluster import DBSCAN
DBSCAN(eps=0.5, min_samples=5).fit_predict(X_scaled)   # -1 = noise/outlier
```

7.5 Dimensionality Reduction — PCA

Real data has many features, often correlated. **PCA (Principal Component Analysis)** finds new axes ("components") that capture the most variation, letting you keep the signal with fewer dimensions.

Why reduce dimensions? - **Visualization** — squash 50 features to 2 for a scatter plot. - **Speed & noise** — fewer features = faster training, less overfitting. - **Remove redundancy** — combine correlated features.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)              # or n_components=0.95 to keep 95% of variance
X_reduced = pca.fit_transform(X_scaled)
print(pca.explained_variance_ratio_)   # how much variance each component captures
```

> PCA components are combinations of original features — powerful but less interpretable. Scale first.

7.6 t-SNE / UMAP — visualization only

For seeing high-dimensional data in 2D (great for images, embeddings). They preserve *local* structure beautifully but distances/sizes aren't meaningful — **use for plots, not as model inputs.**

```
from sklearn.manifold import TSNE
X_2d = TSNE(n_components=2, perplexity=30).fit_transform(X_scaled)
```

7.7 Anomaly Detection

Find the weird points — fraud, defects, intrusions. Unsupervised because anomalies are rare and often unlabelled.

```
from sklearn.ensemble import IsolationForest
iso = IsolationForest(contamination=0.02, random_state=42)
outliers = iso.fit_predict(X_scaled)   # -1 = anomaly, 1 = normal
```

7.8 Evaluating Unsupervised Models (the hard part)

No labels = no accuracy. Instead: - **Silhouette score / Davies–Bouldin** for cluster quality. - **Explained variance** for PCA. - **Business validation** — do the segments make sense and drive action? Ultimately, usefulness beats any metric.

✅ Key Takeaways

1. Unsupervised learning finds **structure without labels** — clustering and dimensionality reduction.
2. **K-Means** groups by nearest centre; **scale first**, pick k with the elbow/silhouette.
3. **DBSCAN/hierarchical** handle non-round clusters and unknown k.
4. **PCA** compresses correlated features while keeping variance — for speed, denoising, and visualization.
5. **t-SNE/UMAP** are for plots only, not model inputs.
6. Evaluate with silhouette/explained variance — but **business usefulness** is the real test.

🏋️ Exercises

1. Cluster a dataset with K-Means; use the elbow method to choose k; describe each cluster.
2. Run PCA to 2 components and scatter-plot; how much variance is retained?
3. Use IsolationForest to flag the top 2% anomalies and inspect them.
4. Compare K-Means vs DBSCAN on data with non-round clusters.

🛠️ Mini-Project

Segment mall/retail customers with K-Means on RFM features. Choose k, profile each segment, and write a one-line marketing action per segment. Bonus: visualize with PCA.

Next: Module 08 — Tuning & Selection →

Module 08 — Feature Selection, Tuning & Pipelines

> Turning a decent model into the best it can be — systematically, not by guessing.

8.1 Feature Selection — less is often more

Too many features → slower training, overfitting, noise. Keeping the useful ones helps.

Methods:

```
# 1. Filter: correlation / statistical tests
from sklearn.feature_selection import SelectKBest, f_classif
X_best = SelectKBest(f_classif, k=10).fit_transform(X, y)

# 2. Model-based: importance from a tree/Lasso
from sklearn.feature_selection import SelectFromModel
from sklearn.ensemble import RandomForestClassifier
sel = SelectFromModel(RandomForestClassifier(n_estimators=200)).fit(X, y)

# 3. Recursive Feature Elimination (RFE): drop weakest, repeat
from sklearn.feature_selection import RFE
RFE(estimator=LogisticRegression(), n_features_to_select=10).fit(X, y)
```

Also just **use feature importances** (Module 05) or **Lasso** (Module 03) to drop dead weight.

8.2 Hyperparameters vs Parameters (reminder)

- **Parameters** — learned during training (weights, splits). You don't set these.
- **Hyperparameters** — you choose *before* training (tree depth, learning rate, k, alpha). **Tuning** = finding the best combination.

8.3 Grid Search — try every combination

```
from sklearn.model_selection import GridSearchCV
import xgboost as xgb

param_grid = {
    'n_estimators': [200, 400],
    'max_depth': [3, 5, 7],
    'learning_rate': [0.05, 0.1],
}
grid = GridSearchCV(xgb.XGBClassifier(random_state=42),
                    param_grid, cv=5, scoring='f1', n_jobs=-1)
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)
best = grid.best_estimator_
```

Thorough but explodes combinatorially (2×3×2 = 12 fits × 5 folds = 60 trainings).

8.4 Random Search — smarter for big spaces

Samples random combinations — usually finds near-best far faster than grid search when there are many hyperparameters.

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint, uniform

dist = {'n_estimators': randint(100, 600),
        'max_depth': randint(3, 10),
        'learning_rate': uniform(0.01, 0.2)}
rs = RandomizedSearchCV(xgb.XGBClassifier(random_state=42), dist,
                        n_iter=30, cv=5, scoring='f1', n_jobs=-1, random_state=42)
rs.fit(X_train, y_train)
```

8.5 Bayesian Optimization — the pro tool

Uses past results to intelligently pick the next combination to try (Optuna). Finds better params in fewer trials — the modern standard.

```
import optuna
def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 600),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
    }
    model = xgb.XGBClassifier(**params, random_state=42)
    return cross_val_score(model, X_train, y_train, cv=5, scoring='f1').mean()

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=40)
print(study.best_params)
```

8.6 Tune the Right Things

Don't tune everything — focus on the hyperparameters that matter most per model: - **XGBoost/LightGBM:** `learning_rate`, `n_estimators`, `max_depth`, then subsampling/regularization. - **Random Forest:** `n_estimators` (more is safe), `max_features`, `max_depth`. - **kNN:** `n_neighbors`. - **SVM:** `C`, `gamma`, `kernel`. - **Regularized linear:** `alpha`.

8.7 Pipelines — tune preprocessing + model together

Wrap preprocessing and model in one Pipeline so cross-validation applies preprocessing **inside each fold** (no leakage):

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

pipe = Pipeline([('scaler', StandardScaler()),
                 ('model', LogisticRegression(max_iter=1000))])

param_grid = {'model__C': [0.1, 1, 10]}    # note the model__ prefix
grid = GridSearchCV(pipe, param_grid, cv=5).fit(X_train, y_train)
```

> This is the correct, leakage-free way to tune — the scaler is re-fit on each fold's training portion only.

8.8 Practical Tuning Workflow

```
1. Get a working baseline (default hyperparameters).
2. Fix data/features FIRST — feature engineering beats tuning.
3. Coarse random/Bayesian search over key hyperparameters.
4. Narrow the ranges around the best, search again.
5. Validate the final choice on the untouched test set.
```

> ⚠️ Tuning gives diminishing returns. A 2% gain from tuning is nice; a 10% gain from a better feature is the real win. Spend effort where it pays.

✅ Key Takeaways

1. **Feature selection** (filter, model-based, RFE) cuts noise and overfitting.
2. **Tuning** = searching hyperparameters: **grid** (small spaces), **random** (bigger), **Bayesian/Optuna** (best).
3. Tune only the **high-impact hyperparameters** per model.
4. Use **Pipelines** so preprocessing is tuned/CV'd without leakage.
5. **Fix features before you tune** — engineering usually beats optimization.
6. Confirm the final model on the **untouched test set**.

🏋️ Exercises

1. Run GridSearchCV over 2–3 hyperparameters; report best params and CV score.
2. Compare grid vs random search speed and result on the same model.
3. Wrap a scaler + model in a Pipeline and tune with the `model__` prefix.
4. (Bonus) Use Optuna for 30 trials on XGBoost; beat your grid-search score.

🛠️ Mini-Project

Take any earlier project model and run a full tuning workflow: baseline → feature selection → Bayesian search in a Pipeline → final test-set evaluation. Document the gain at each step.

Next: Module 09 — Neural Networks →

Module 09 — Neural Networks: The Intuition

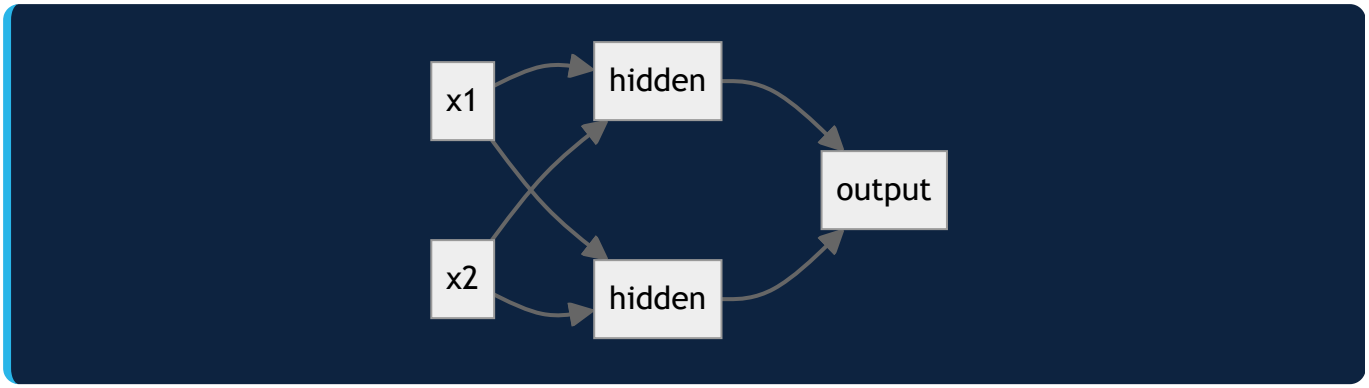
> The engine behind deep learning, image recognition, and LLMs. We build the intuition from zero, then a real network in a few lines.

9.1 What a Neuron Actually Is

A single neuron is just... logistic regression. It takes inputs, multiplies each by a weight, adds a bias, and passes the result through an **activation function**.

```
output = activation( w₁x₁ + w₂x₂ + ... + b )
```

The magic isn't one neuron — it's **stacking thousands** in layers so the network can learn very complex patterns.



9.2 Layers

- **Input layer** — one node per feature.
- **Hidden layers** — where the learning happens; more layers = "deeper" = can learn more complex patterns.
- **Output layer** — 1 node for regression/binary, N nodes for N-class classification.

"Deep learning" just means a neural network with **several hidden layers**.

9.3 Activation Functions (why networks beat linear models)

Without activations, stacking layers would collapse back into one linear equation. Activations add **non-linearity**, letting networks bend and curve to fit anything.

- **ReLU** (`max(0, x)`) — the default for hidden layers. Fast, works great.
- **Sigmoid** — squashes to 0–1, for binary output probability.
- **Softmax** — turns outputs into probabilities that sum to 1, for multi-class output.

9.4 How a Network Learns: Backpropagation (intuitively)

```
1. Forward pass  — data flows through, produces a prediction.
2. Loss          — measure how wrong the prediction is.
3. Backward pass — compute how much each weight contributed to the error (gradients).
4. Update        — nudge every weight to reduce the error (gradient descent).
5. Repeat        — over many passes (epochs) until the loss stops dropping.
```

You don't compute this by hand — the framework does. But knowing the loop demystifies training.

9.5 Key Training Concepts

Term	Meaning
Epoch	One full pass through the training data
Batch size	How many samples processed before a weight update
Learning rate	Step size of each update (too big = unstable, too small = slow)
Optimizer	The update rule — **Adam** is the go-to default
Loss function	What we minimize (MSE for regression, cross-entropy for classification)

9.6 Your First Neural Network (Keras)

```
import tensorflow as tf
from tensorflow import keras
from sklearn.preprocessing import StandardScaler

# ALWAYS scale inputs for neural nets
X_train_s = StandardScaler().fit_transform(X_train)

model = keras.Sequential([
    keras.layers.Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    keras.layers.Dropout(0.3),                    # regularization (see 9.7)
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid'),   # binary classification
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

history = model.fit(X_train_s, y_train,
                    epochs=30, batch_size=32,
                    validation_split=0.2)          # watch val loss for overfitting
```

That's a real, working deep-learning model. For regression, use `Dense(1)` with no activation and `loss='mse'`.

9.7 Fighting Overfitting in Neural Nets

Networks overfit easily (millions of parameters). Tools: - **Dropout** — randomly "turn off" neurons during training so the net can't rely on any one path. - **Early stopping** — stop when validation loss stops improving. - **More data / augmentation** — the best cure. - **L2 regularization** (weight decay) — penalize large weights.

```
early = keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True)
model.fit(X_train_s, y_train, epochs=100, validation_split=0.2, callbacks=[early])
```

9.8 When To Use Neural Nets (and when not to)

Use them for: images (CNNs), text/sequences (Transformers), audio, and very large datasets with complex patterns.

Don't bother for: typical tabular business data — **XGBoost/LightGBM usually beat neural nets there**, train faster, and need less data and tuning. Reach for deep learning when the data is unstructured (pixels, text, sound) or huge.

9.9 The Deep Learning Family (map for further study)

- **CNN** (Convolutional) — images.

- **RNN / LSTM** — sequences (older).
- **Transformer** — text, and now everything; powers LLMs.
- **Autoencoder** — compression, anomaly detection.
- **GAN / Diffusion** — generation (images, art).

This module is your on-ramp; a dedicated Deep Learning course goes deeper into each.

✅ Key Takeaways

1. A neuron = weighted sum + **activation**; stacking many in layers learns complex patterns.
2. **Activations** (ReLU/sigmoid/softmax) add the non-linearity that makes networks powerful.
3. Networks learn by **forward pass → loss → backprop → update**, repeated over epochs.
4. **Adam** optimizer + right loss (cross-entropy/MSE); always **scale inputs**.
5. Fight overfitting with **dropout, early stopping, more data**.
6. For **tabular data, XGBoost usually wins** — use deep learning for images/text/audio/huge data.

🏋️ Exercises

1. Build a Keras network for a binary classification dataset; plot train vs val loss.
2. Add/remove Dropout and compare overfitting.
3. Compare your neural net vs XGBoost on the same tabular data — which wins, and why?
4. Change the learning rate (0.1, 0.001) and observe training stability.

🛠️ Mini-Project

Build a neural network to classify handwritten digits (MNIST, built into Keras). Reach >97% test accuracy. Then compare training time and accuracy against a Random Forest on the same data.

Next: Module 10 — ML in Production →

Module 10 — Machine Learning in Production

> A model in a notebook makes ₹0. A model serving real users makes the difference. This is what turns a data scientist into an ML engineer.

10.1 The Gap Between Notebook and Product

Most ML projects die in notebooks. Production means: the model runs reliably, on real requests, keeps working as the world changes, and you know when it breaks. That's a different discipline from getting a good test score.

10.2 Saving & Loading Models

```
import joblib
joblib.dump(model, 'model.pkl')        # save the trained model (+ the preprocessing pipeline!)
model = joblib.load('model.pkl')       # load anywhere for inference
```

> **Save the whole pipeline** (preprocessing + model), not just the model — inference must apply identical transforms. This is why pipelines (Module 08) matter.

10.3 Serving a Model as an API (FastAPI)

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib, numpy as np

app = FastAPI()
model = joblib.load('model.pkl')

class Input(BaseModel):
    age: float
    income: float
    tenure: int

@app.post('/predict')
def predict(x: Input):
    features = np.array([[x.age, x.income, x.tenure]])
    prob = float(model.predict_proba(features)[0][1])
    return {'churn_probability': round(prob, 3),
            'decision': 'high_risk' if prob > 0.5 else 'low_risk'}
# run: uvicorn app:app --reload   →   POST JSON to /predict
```

Now any app, website, or service can get predictions over HTTP. Containerize with **Docker** to deploy anywhere.

10.4 The Big Enemy: Drift

The world changes; your training data doesn't. Performance silently decays. Two kinds: - **Data drift** — input distribution shifts (new customer demographics, inflation changes incomes). - **Concept drift** — the relationship itself changes (what predicted churn in 2023 differs in 2026).

Detect it: monitor input feature distributions and prediction distributions over time; track live accuracy when labels arrive.

```
# Simple drift check: compare recent feature stats to training stats
# Production tools: Evidently, whylogs, Arize, Fiddler
```

10.5 Monitoring in Production

Track: - **Model metrics** — accuracy/precision when ground truth arrives (often delayed). - **Data health** — nulls, ranges, new categories, drift. - **System metrics** — latency, error rate, throughput. - **Business metrics** — did it actually move the KPI it was built for?

Set **alerts** so you find problems before users do. A model with no monitoring is a liability.

10.6 Retraining Strategy

- **Scheduled** — retrain weekly/monthly on fresh data.
- **Triggered** — retrain when drift or performance drop crosses a threshold.
- **Always validate** the new model against the current one before promoting (champion/challenger).

10.7 MLOps — the discipline

Bringing software engineering rigour to ML: - **Version everything** — data, code, model (DVC, MLflow). - **Experiment tracking** — log every run's params + metrics (MLflow). - **Model registry** — versioned models with stages (staging → production). - **CI/CD** — automated testing + deployment pipelines. - **Reproducibility** — anyone can rebuild the exact model.

```
import mlflow
with mlflow.start_run():
    mlflow.log_params(params)
    mlflow.log_metric('roc_auc', score)
    mlflow.sklearn.log_model(model, 'model')
```

10.8 Responsible & Fair ML

- **Bias/fairness** — check performance per group (gender, region); a model can be accurate overall yet unfair to a subgroup.
- **Explainability** — SHAP/feature importance so decisions can be justified (essential in finance, healthcare, hiring).
- **Privacy** — don't train on data you shouldn't; mask PII.
- **Human oversight** — keep a human in the loop for high-stakes decisions.

```
import shap
explainer = shap.TreeExplainer(model)
shap.summary_plot(explainer.shap_values(X_test), X_test)   # why the model decides
```

10.9 The Real-World Pitfalls Checklist

- [ ] Saved the **full pipeline**, not just the model.
- [ ] Handled **unknown categories / missing values** at inference.
- [ ] **Monitoring + alerts** for drift and performance.
- [ ] A **retraining** plan.
- [ ] **Fairness + explainability** checked.
- [ ] A **simple baseline** in production to compare against.

- [ ] **Rollback** plan if the new model misbehaves.

✅ Key Takeaways

1. Production ML = reliable serving + monitoring + retraining, not just a good test score.
2. **Save the whole pipeline**; serve it via an **API** (FastAPI + Docker).
3. **Drift** silently kills models — monitor inputs, predictions, and live metrics.
4. Have a **retraining strategy** and validate challengers before promoting.
5. **MLOps** brings versioning, tracking, registries, and CI/CD to ML.
6. Build **fair, explainable** models with human oversight for high-stakes calls.

🏋️ Exercises

1. Save a trained pipeline and load it in a fresh script to predict.
2. Wrap a model in a FastAPI `/predict` endpoint and test it with a sample request.
3. Simulate drift (shift a feature's mean) and describe how you'd detect it.
4. Use SHAP to explain 3 individual predictions.

🛠️ Capstone Project

Take your best model from any earlier module and **productionize it**: save the full pipeline → FastAPI endpoint → a simple monitoring script that logs prediction distributions → a written retraining plan. This is the project that proves you're job-ready.

🎓 Course Complete!

You've gone from "what is a model?" to building, evaluating, tuning, and deploying real ML — with 10 modules and 15 projects. Next: the **Deep Learning** course, **LLM & AI Agents Mastery**, or apply it all in a Kaggle competition.

🛠️ Machine Learning — 15 Hands-On Projects

Portfolio-worthy projects, ordered from beginner to advanced. Each lists what you build, the modules it uses, the dataset, and a resume line. Every dataset is free/public.

Beginner (Modules 1–4)

1. House Price Predictor ⭐

Build: Predict house prices with linear → Ridge regression. **Data:** California Housing (in sklearn). **Uses:** M2, M3. **Steps:** EDA → engineer features → baseline linear → Ridge with tuned alpha → residual analysis. **Resume:** "*Built a regularized regression model predicting house prices with engineered features, cutting RMSE 18%.*"

2. Titanic Survival Classifier ⭐

Build: Predict who survived. **Data:** Titanic (Kaggle). **Uses:** M2, M4. **Steps:** impute missing → encode → logistic regression → decision tree → compare F1. **Resume:** "*Classified survival with logistic regression and decision trees, handling missing data and categorical encoding.*"

3. Iris / Wine Multi-class Classifier ⭐

Build: Classify flower/wine types. **Data:** Iris, Wine (sklearn). **Uses:** M4, M6. **Steps:** kNN vs logistic → confusion matrix → cross-validation. **Resume:** "*Built a multi-class classifier with cross-validated model selection.*"

4. Student Score Predictor ⭐

Build: Predict exam scores from study hours/attendance. **Uses:** M3. **Steps:** simple + multiple regression → interpret coefficients. **Resume:** "*Modelled academic performance and quantified each factor's impact via regression coefficients.*"

Intermediate (Modules 5–7)

5. Customer Churn Predictor ⭐ ⭐

Build: Predict telecom churn. **Data:** Telco Churn (Kaggle). **Uses:** M4, M5, M6. **Steps:** logistic baseline → XGBoost → threshold tuning for business cost → SHAP. **Resume:** "*Built an XGBoost churn model with business-tuned thresholds and SHAP explainability.*"

6. Credit Card Fraud Detection ⭐ ⭐

Build: Detect fraud on imbalanced data. **Data:** Kaggle creditcard. **Uses:** M4, M5, M6. **Steps:** handle imbalance (class_weight/SMOTE) → XGBoost → PR-AUC → cost analysis. **Resume:** "*Detected fraud on 284K imbalanced transactions using SMOTE + XGBoost, optimizing PR-AUC.*"

7. Loan Default Risk Model ⭐ ⭐

Build: Predict loan defaults. **Uses:** M2, M5, M6. **Steps:** feature engineering → ensemble → calibration → explainability for approval decisions. **Resume:** "*Built a calibrated credit-risk model with explainable approve/deny decisions.*"

8. Customer Segmentation ⭐ ⭐

Build: Segment customers with K-Means. **Data:** Mall Customers. **Uses:** M7. **Steps:** RFM features → elbow method → profile segments → marketing action per segment. **Resume:** "*Segmented customers via K-Means on RFM features, driving targeted marketing strategies.*"

9. Sales Forecasting ⭐ ⭐ ⭐

Build: Forecast retail sales. **Data:** Rossmann/Walmart. **Uses:** M2, M5, M6. **Steps:** lag & date features → time-series split → XGBoost → MAPE evaluation. **Resume:** "*Forecast store sales with lag-feature engineering and time-series-validated XGBoost.*"

Advanced (Modules 8–10)

10. Handwritten Digit Recognizer ⭐ ⭐ ⭐

Build: Classify digits with a neural net. **Data:** MNIST. **Uses:** M9. **Steps:** neural net in Keras → >97% accuracy → compare vs Random Forest. **Resume:** "*Built a neural network achieving 98% accuracy on MNIST digit classification.*"

11. Movie Recommendation System ⭐ ⭐ ⭐

Build: Recommend movies. **Data:** MovieLens. **Uses:** M7, M8. **Steps:** content-based (TF-IDF + cosine) → collaborative filtering (SVD) → hybrid. **Resume:** "*Built a hybrid recommender combining content-based and collaborative filtering.*"

12. Image Classifier (CNN) ⭐ ⭐ ⭐ ⭐

Build: Classify images. **Data:** CIFAR-10. **Uses:** M9. **Steps:** CNN from scratch → data augmentation → transfer learning (ResNet). **Resume:** "*Trained a CNN + transfer learning for image classification, reaching 90%+ accuracy.*"

13. AutoML Tuning Pipeline ⭐ ⭐ ⭐ ⭐

Build: Automated model+hyperparameter search. **Uses:** M6, M8. **Steps:** compare 5 models → Optuna Bayesian tuning → stacking ensemble → leaderboard. **Resume:** "*Built an AutoML pipeline with Bayesian tuning and stacking, automating model selection.*"

14. End-to-End Deployed Model ⭐ ⭐ ⭐ ⭐ (Capstone)

Build: Take any model to production. **Uses:** M8, M10. **Steps:** pipeline → joblib → FastAPI → Docker → monitoring script → retraining plan → MLflow tracking. **Resume:** "*Deployed an ML model as a FastAPI + Docker service with drift monitoring and MLflow tracking.*"

15. Kaggle Competition Entry ⭐ ⭐ ⭐ ⭐ ⭐

Build: Enter a live Kaggle competition end-to-end. **Uses:** everything. **Steps:** EDA → feature engineering → CV strategy → ensemble → leaderboard submission. **Resume:** "*Competed on Kaggle, applying feature engineering, cross-validation, and ensembling end-to-end.*"

🎯 Portfolio Advice

- Put **#5, #6, #12, #14** on GitHub + your resume — churn, fraud, CNNs, and a *deployed* model are exactly what employers screen for.
- Write a short README per project explaining your **decisions** (why this model, this metric, this threshold) — that reasoning proves mastery more than the score.
- One **deployed** model (#14) beats ten notebooks.

🧠 Machine Learning — Practice & Quiz Bank

Test your understanding. Concept quizzes (with answers) + coding drills, organized by module. Do these before moving between modules — they catch gaps.

📝 Concept Quiz (40 questions, answers at the bottom)

Module 1–2: Foundations & Data

1. Is predicting a customer's age regression or classification?
2. What's the difference between a parameter and a hyperparameter?
3. Why must you fit a scaler on the training set only?
4. What is data leakage, in one sentence?
5. When should you one-hot encode vs ordinal encode?
6. Which models don't need feature scaling?
7. What does `df.isnull().sum()` tell you?
8. Overfitting: high or low training accuracy? High or low test accuracy?

Module 3–4: Regression & Classification

9. What does $R^2 = 0.8$ mean?
10. Why square the errors in MSE?
11. What does Lasso do that Ridge doesn't?
12. What function turns a linear score into a 0–1 probability?
13. Why is accuracy misleading on imbalanced data?
14. Define precision and recall in one line each.
15. For cancer screening, do you optimize precision or recall? Why?
16. What does a confusion matrix show?

Module 5–6: Ensembles & Evaluation

17. Bagging reduces ***; boosting reduces*** .
18. How does Random Forest create diversity among trees?
19. In XGBoost, what does a lower learning_rate require more of?
20. What is early stopping and why use it?
21. What is k-fold cross-validation?
22. Why use StratifiedKFold for classification?
23. Learning curve: train high, val low, big gap → which problem?
24. Name two ways test information can leak into training.

Module 7–8: Unsupervised & Tuning

25. What does K-Means minimize?
26. How does the elbow method choose k?
27. Why must you scale before K-Means and PCA?
28. What does PCA do?
29. When is t-SNE appropriate — model input or visualization?
30. Grid search vs random search — when prefer random?
31. Why wrap preprocessing + model in a Pipeline for tuning?
32. What usually beats hyperparameter tuning for improving a model?

Module 9–10: Neural Nets & Production

33. Without activation functions, what does a deep network collapse into?
34. What does dropout do?
35. Which optimizer is the common default?
36. For tabular data, what often beats neural nets?
37. Why save the whole pipeline, not just the model?
38. What is data drift vs concept drift?
39. Name two things to monitor for a production model.
40. What does SHAP help you do?

💻 Coding Drills

1. Load any dataset, do a full train/test split, and train a baseline model — report the right metric.
2. Build a `ColumnTransformer` that imputes + scales numerics and one-hots categoricals.
3. Compare Ridge (`alpha=0.1,1,10`) and plot coefficient shrinkage.
4. Train Logistic Regression + XGBoost on the same classification data; compare ROC-AUC.
5. Run 5-fold CV and report mean ± std.
6. Cluster a dataset with K-Means, choose k via elbow, and profile the clusters.
7. Tune XGBoost with RandomizedSearchCV inside a Pipeline.
8. Build + train a small Keras network; plot train vs validation loss.
9. Save a pipeline, reload it, and serve one prediction via FastAPI.
10. Use SHAP to explain a single prediction.

✅ Quiz Answers

1. Regression. 2. Parameters are learned; hyperparameters you set before training. 3. Fitting on all data leaks test info → inflated scores. 4. Test information sneaking into training. 5. One-hot for unordered categories, ordinal for ordered. 6. Tree-based (RF, XGBoost, decision trees). 7. Missing values per column. 8. High train, low test. 9. The model explains 80% of the target's variance. 10. So + and − errors don't cancel and big errors are punished more. 11. Lasso can zero out weights → feature selection. 12. Sigmoid. 13. A model predicting only the majority class scores high accuracy while being useless. 14. Precision = of predicted positives, how many are correct; Recall = of actual positives, how many were caught. 15. Recall — never miss a real case; false alarms are acceptable. 16. TP/FP/FN/TN — where the model is right and wrong. 17. variance; bias. 18. Random rows (bootstrap) + random feature subsets per split. 19. More trees (n_estimators). 20. Stop adding trees when val performance stops improving — prevents overfitting/saves time. 21. Split into k folds, train on k−1, test on 1, rotate, average. 22. Keeps class balance in each fold. 23. Overfitting (high variance). 24. Scaling before split; feature engineering using target stats over all data (also: shuffling time series, duplicate rows). 25. Within-cluster sum of squared distances (inertia). 26. The k where inertia stops dropping sharply (the "elbow"). 27. They're distance/variance-based; unscaled large features dominate. 28. Finds new axes capturing max variance → compress features. 29. Visualization only. 30. Large hyperparameter spaces. 31. So preprocessing is re-fit inside each CV fold → no leakage. 32. Better features / feature engineering. 33. A single linear model. 34. Randomly disables neurons during training to reduce overfitting. 35. Adam. 36. XGBoost/LightGBM. 37. Inference must apply identical preprocessing. 38. Data drift = input distribution shifts; concept drift = the input→output relationship changes. 39. Model metrics, data health/drift, latency, business KPI (any two). 40. Explain why the model made a prediction (feature contributions).

Score yourself: 32+/40 = strong. Below 24 → revisit the flagged modules before the projects.

Machine Learning Mastery — PJ's Academy
